

exp-mlkem-f203-pke-keygen-k4

FIPS 203 Alg.13 ML-KEM-768 PKE KeyGen 实现方案 (k=4)

ascendc 工程 (交付示例)

2026-06-29

摘要

本文档描述 **交付示例** `examples/incubating/exp-mlkem-f203-pke-keygen-k4/` 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 13** (ML-KEM-768, $K = 4$) **PKE KeyGen**。

自包含原则: AscendC 与 Host golden 源码 **全部 vendored** 于本目录 (`prep/`、`compute/`、`scripts/prep/`、`scripts/compute/`)。唯一允许的外部代码依赖: 仓库 `library/shared/` (SHAKE XOF、Keccak 标量核等 AscendC 共享库)。**禁止**: 从 `ascendc-tests/`、`thirdparty/`、`frozen/`、`liboqs` 等 **非 shared** 库引入编译期 `include` 或 Host 运行时接口。

生产 I/O: `input/` 仅 SEED_D + NTT LUT; `output/` 仅 `ek_PKE(1568B)/dk_PKE(1536B)`。设备 **Launch: 2 次**——Launch 1 行 3–15(`f203_keygen_prep`); Launch 2 行 16–21(`mmad_custom` + 行 21 $\parallel \rho$ 设备融合)。中间 GM **不落盘**。Prep **双 AIV 并行** `BuildAHat16ShardWithUb` (`blockIdx 0/1` 各 8 poly); **禁止** pass 探针 `block0` 串行两片 $\hat{A}$ workaround。SIM 910B4 参考 Total tick ≈ 542339 (`prep` $\approx 462k$ + `mmad` $\approx 80k$)。唯一交付路径: 全链向量化 AscendC + 生产 I/O; **不存在** G0–G4 分段门禁、标量回退、磁盘 staging 或多 Host 入口。技术总结: `docs/notes/F203-KeyGen-prep双AIV与SHAKE内嵌技术总结.md`、`docs/notes/F203-KeyGen-prep-Pipe细同步技术总结.md`。

目录

1 工程约束 (自包含)	2
1.1 允许的外部依赖	2
1.2 禁止的外部依赖	2
1.3 唯一交付路径 (向量化全链)	2
1.4 目录布局 (源码归属)	3
2 数学语义 (Alg.13, $K = 4$, $N = 256$, $q = 3329$)	3
2.1 输入输出	3
2.2 流水线	3
3 编排与 Launch	4
4 AI Core 规模	4

目录	2
5 数据与组件同步规划	4
5.1 实现映射与 include 链（本目录 vendored）	4
6 验证	5
6.1 生产验收（默认）	5
6.2 可选 Host golden 对拍	5
6.3 liboqs PKE KeyGen KAT（交付前必跑，非 run.sh 默认）	5
7 性能（SIM 910B4 参考，2026-06-29）	5

1 工程约束（自包含）

1.1 允许的外部依赖

路径	用途
library/shared/shake_x of_kernel/	AscendC SHAKE XOF 内嵌 (ProcessInline)
library/shared/keccak_ f1600_kernel/	Keccak-f[1600] 标量 (G 派生, 第三方 SHA3 到位前)
library/shared/	公共头 (如 shake_general.h)
CANN / tikicpulib	工具链与仿真 (非业务源码)

1.2 禁止的外部依赖

- **ascendc-tests** 任意探针目录的 *.hpp/*.cpp include 或 Python import。
- **thirdparty/merged_kyber**、polybatch 探针、vec-k4-v2 外链。
- **liboqs** 作为 **编译期/运行时依赖** (仅 kat_liboqs_vs_ascendc.sh 可选 KAT 对照, 非 run.sh 契约)。
- 分段 Host/设备入口、KEYGEN_GATE、中间 bin staging、Host 拼接 ek|| (行 21 已在 compute 内核融合)。

1.3 唯一交付路径（向量化全链）

本示例 仅下列一条路径；run.sh 内 CMake 选项 **硬编码**，不提供标量/分段/legacy 开关：

组件	锁定配置（与 run.sh 一致）
Prep Â	F203_AHAT16_BLOCK_DIM=2, 双 AIV 并行 BuildAHat16ShardWithUb (blockIdx 分片), 内嵌 SHAKE ProcessInline
Prep Alg7	F203_ALG7_REJ_IMPL=1, F203_ALG7_D12_GATHER=0
Prep CBD	prep/alg8/f203_cbd_eta2_ub_io.hpp 向量 MTE2/V
Compute 2s1e	F203_STAGE1_SPLIT=1, mixPass=0
Compute NTT/Alg11	HAT_ALG11_VEC=1, ALG11_IMPL=1, ALG11_VEC_VARIANT=2, ALG11_MEM_OPS=1
行 18-20	HAT_LINE18_DOT_ONLY=0, HAT_BYTE_ENCODE=1
ByteEncode	BYTE_ENCODE12_VEC=1, scatter+prefetch
行 21	F203_KEYGEN_EK_PKE=1 (设备 ek , 无单独 launch)

Host	main_keygen.cpp + ascendc_keygen_bbit, 2 launch, seed+LUT→ek/dk
------	--

禁止: G0-G4 门禁脚本、main_keygen_prep/f203_keygen_ek_append 分段入口、compute_io/磁盘 staging、mixPass≠0 调试路径作为交付验收。

1.4 目录布局 (源码归属)

exp-mlkem-f203-pke-keygen-k4/

main_keygen.cpp	# 生产 Host: 2 launch
f203_keygen_prep_entry.cpp	# Launch 1 入口
f203_keygen_prep_ub.hpp	# prep 编排 (P-02/03/04)
prep/	# vendored 行 3-15 (ahat/alg7/presample/alg8)
compute/	# vendored 行 16-21 (mmad_custom + ek rho 融合)
cmake/keygen/CMakeLists.txt	# 生产构建 (仅 2-launch 生产路径)
scripts/prep/	# Alg7 ROM + golden 准备段
scripts/compute/	# compute golden + LUT 生成
scripts/keygen_golden.py	# Host 全链 golden (自包含 import)
scripts/prepare_production_input.py	
scripts/verify_production.py	
scripts/kat_liboqs_vs_ascendc.py	
thirdparty/ntt_study/	# Host golden LUT 表 (vendored, 非 AscendC 编译依赖)
run.sh	
kat_liboqs_vs_ascendc.sh	# 可选 liboqs PKE KeyGen KAT (CPU×10 + SIM×1)
SELF_CONTAINED.md	

2 数学语义 (Alg.13, $K = 4$, $N = 256$, $q = 3329$)

2.1 输入输出

- **输入** input/seed_d.bin: 4 B uint32 LE; Host 写入 GM; 约定 DerandFromSeedD。
- **输入** input/lut_even_stacked.bin, lut_odd_stacked.bin: NTT LUT(静态;缺失时 scripts/compute/g 生成一次)。
- **输出** output/ek_pke.bin: 1568 B = $\text{ByteEncode}_{12}(t) \parallel \rho$ 。
- **输出** output/dk_pke.bin: 1536 B = $\text{ByteEncode}_{12}(s)$ 。
- **禁止:** 向 input/ 写入 a_hat、src、rho、tiling 等; 向 output/ 写入除 ek/dk 外的交付文件 (KEYGEN_DEBUG_DUMP=1 除外)。

2.2 流水线

1. $G(d||k) \rightarrow (\rho, \sigma)$ 。

2. 行 3-7: $16 \times \text{SampleNTT}(\rho) \rightarrow \hat{A}$ 。
3. 行 8-15: $\text{PRF} + \text{CBD}_{\eta_2} \rightarrow \hat{s}$ 。
4. 行 16-20: `compute/mmad_custom` (NTT、内积、`ByteEncode12`)。
5. 行 21 (设备 AIV): `ek_PKE = ek_polyvec ||  $\rho$` , `F203_KEYGEN_EK_PKE=1`, 与行 16-20 同次 Launch 2。

3 编排与 Launch

- 用户入口: `bash run.sh -r cpu|sim -v Ascend910B4`。
- **Launch 1:** `f203_keygen_prep (KERNEL_TYPE_AIV_ONLY, blockDim=2)`, GM 输出 $\hat{A}$ 、 $\hat{s}$ 、 ρ (及 PRF 中间态, 默认不落盘)。
- **Launch 2:** `compute/mmad_custom (KERNEL_TYPE_MIX_AIC_1_2, blockDim=1)`, 读 prep GM + LUT; 写 `ek_PKE/dk_PKE`。
- **Host:** `prepare_production_input` $\rightarrow$ `build` $\rightarrow$ 单次 `ascendc_keygen_bbit` $\rightarrow$ `scrub output/`。
- **SIM:** `build` 后 `sim_env_export (WSL stub)` $\rightarrow$ `camodel_sim_log` $\rightarrow$ `kernel` (顺序与探针一致)。

4 AI Core 规模

段	核类型 / blockDim / SIM 实际
Prep	AIV_ONLY; blockDim=2; 双 AIV 并行 $\hat{A}$ (各 8 poly); block0 独占 PRF+CBD
Compute	MIX_AIC_1_2; blockDim=1; 1 AIC + 2 AIV

串行占用 **1 颗 AI Core**。CPU [SUCCESS] [AIC_*] 为 tikicpu 拓扑 artifact; 以 `sim_log/profile_*.toml` 为准。

5 数据与组件同步规划

prep 单内核内 UB/GM 同步; prep 与 compute 之间仅 Host 顺序 launch + GM 指针传递 (无文件交接)。原理与 Pipe 细同步见 `docs/notes/F203-KeyGen-prep双AIV与SHAKE内嵌技术总结.md`、`docs/notes/F203-KeyGen-prep-Pipe细同步技术总结.md`。

5.1 实现映射与 include 链 (本目录 vendored)

`f203_keygen_prep_entry.cpp`

$\rightarrow$ `f203_keygen_prep_ub.hpp`

(P-02/P-03/P-04)

```
-> prep/ahat/f203_a_hat16_ub.hpp      (A-*)
-> prep/presample/f203_se_vector_prf.hpp (R-*)
-> prep/alg8/f203_cbd_eta2_ub_io.hpp (C-02/C-03/C-04)
-> library/shared/shake_xof_kernel    (S-01, ProcessInline)
```

main_keygen.cpp

```
-> Launch 1: f203_keygen_prep
-> Launch 2: compute/mmad_custom.cpp (F203_KEYGEN_EK_PKE=1)
```

CMake (cmake/keygen/CMakeLists.txt) 仅设置 COMPUTE_ROOT、PREP_* 为本目录子路径 + library/shared; 不得再出现 VEC_V2、AHAT_INC 等外链变量。

6 验证

6.1 生产验收 (默认)

```
cd examples/incubating/exp-mlkem-f203-pke-keygen-k4
bash run.sh -r cpu -v Ascend910B4
bash run.sh -r sim -v Ascend910B4 # 默认即全量; run.sh 内自动 SIM_DIRECT=1
bash kat_liboqs_vs_ascendc.sh
# 检查 output/ek_pke.bin (1568B) + dk_pke.bin (1536B)
```

禁止将 SIM_DIRECT=1、HAT_*=0/1 等写入标准验收命令 (与 run.sh 默认重复)。

6.2 可选 Host golden 对拍

KEYGEN_VERIFY=1 bash run.sh -r cpu: KEYGEN_GOLDEN_ONLY=1 仅写 golden_ek/dk, 不污染 input/。

6.3 liboqs PKE KeyGen KAT (交付前必跑, 非 run.sh 默认)

```
bash kat_liboqs_vs_ascendc.sh
# 期望: CPU 10/10 OK; SIM 1/1 OK (stdout 仅计数, 失败时 log 含 seed_d)
```

与 ascendc-tests/pass-fix-f203-alg13-device-keygen-k4/ 同脚本契约; liboqs 0.15.0 PKE ref 静态链。

7 性能 (SIM 910B4 参考, 2026-06-29)

Total tick $\approx$ 542339 (prep $\approx$ 462161 + mmad $\approx$ 80475; 单进程 2 launch; profile_subtask_log*.toml 求和)。相对 pass 探针 block0 串行 $\hat{A}$ ($\approx$ 886801 tick), prep 双 AIV 并行 fork 为主要增益。CPU [SUCCESS] [AIC_*] 为 tikicpu artifact; 以 SIM toml 为准。